HANDS-ON LAB GUIDE

Data Quality & Governance Platform

13 Hands-On Labs · Beginner to Advanced · Full Step-by-Step Instructions

Lab	Title	Time	Level
Lab 00	Environment Setup & Platform Tour	30 min	■ Beginner
Lab 01	Domains, Subdomains & Data Assets	45 min	■ Beginner
Lab 02	Creating & Running Data Quality Rules	60 min	■ Beginner
Lab 03	Scheduling & Automated Execution	40 min	■■ Intermediate
Lab 04	AI Copilot — Rule Wizard & NL Rules	50 min	■■ Intermediate
Lab 05	Data Catalog, Glossary & Column Profiling	55 min	■■ Intermediate
Lab 06	Governance Scorecards & Policy Engine	60 min	■■ Intermediate
Lab 07	Compliance Frameworks & Data Contracts	60 min	■■■ Advanced
Lab 08	Privacy Engineering & Masking Policies	45 min	■■■ Advanced
Lab 09	Incident Management & Root Cause Analysis	55 min	■■■ Advanced
Lab 10	Data Lineage & Impact Analysis	50 min	■■■ Advanced
Lab 11	Rule Marketplace & Template Publishing	40 min	■■ Intermediate
Lab 12	API Automation & CI/CD Integration	60 min	■■■ Advanced

v3.0 · May 2026 · ~10.5 hours total lab time · All labs self-contained

How to Use This Guide

Each lab is self-contained and can be completed independently. Labs 00–03 form the required foundation. Each lab builds on real platform features — not simulations.

Symbol	Meaning
■ Code block	Command to type exactly as shown in a terminal
■ Navigation	UI path to follow (e.g. Rules → New Rule)
■ Expected result	What you should see after completing the step
■■ Warning	Common mistakes or destructive actions
■ Tip	Shortcut or pro tip to save time
■ Note	Background information or context

Prerequisites

- Python 3.12+, Node.js 22+, Docker Desktop installed and running
- A Snowflake account (required for rule execution in Labs 02+; all other features work without it)
- Git to clone the repository
- 10–15 GB free disk space (for Docker images and Ollama model)

■ **Note:** Labs 00–04 work without Snowflake. Rule executions return an error, but catalog, governance, AI chat, and all other features function fully.

LAB 00

Environment Setup & Platform Tour

■ 30 min ■ ■ Beginner

■ Lab Objectives

- Install and start all platform services using Docker Compose
- Access the web UI and navigate all 9 sidebar sections
- Confirm backend health endpoint returns OK
- Configure the LLM provider (Ollama or OpenAI)
- Verify seed data: 7 domains, 32 subdomains

Background

The platform runs three services: FastAPI backend (port 8000), Next.js frontend (port 3000), PostgreSQL (port 5432). Docker Compose starts all three. On first startup the backend creates all 47 database tables and seeds 7 business domains.

Part 1: Clone and Start

Step 1: Clone the repository

- Open a terminal window
- git clone <your-repo-url> data-quality-app
- cd data-quality-app

Step 2: Configure environment variables

- cp .env.example .env
- Generate SECRET_KEY: python -c "import secrets; print(secrets.token_hex(32))"
- Paste the output as SECRET_KEY= in .env
- Set AUTH_REQUIRED=false for local development
- Optionally set LLM_PROVIDER=ollama

Step 3: Start all services

- docker compose up -d
- Wait 20 seconds, then: docker compose ps (all should show 'Up')
- docker compose logs api | grep -i 'startup complete'

Step 4: Verify health

- curl http://localhost:8000/health
- ■ Expected: {"status": "healthy", "checks": {"database": "ok"}}
- Open http://localhost:3000 in your browser
- ■ Expected: Global Dashboard loads with 7 domain cards

■ **Tip:** The first startup takes 30–60 seconds. If the frontend shows 'API unavailable', wait another 10 seconds and refresh.

Part 2: Configure LLM (Optional but recommended)

Step 5: Set up Ollama (free, local)

- macOS: brew install ollama
- Linux: curl -fsSL https://ollama.ai/install.sh | sh
- ollama pull qwen2.5:7b-instruct (downloads ~4 GB)
- ollama serve (starts on port 11434)

Step 6: Connect LLM in Settings

- Go to: Administration → Settings → LLM / AI
- Provider: Ollama (Local), Base URL: http://localhost:11434
- Model: qwen2.5:7b-instruct
- Click 'Test LLM' — wait 10 seconds for first inference
- ■ Expected: Green 'LLM is working' banner appears

Part 3: Explore Navigation

Step 7: Navigate the 9 sidebar sections

- Click through: Overview, Data Quality, Operations, Catalog, Governance, Privacy, AI Intelligence, Support, Administration
- Click ■ button at top of sidebar — collapses to icon-only mode (click again to expand)
- Press ■K (Mac) or Ctrl+K (Windows) — command palette opens
- Type 'rules' in the command palette and press Enter to navigate to Rules
- Notice: Administration section is only visible to the admin role

■ **Lab 00 Complete!** Your environment is running. Proceed to Lab 01.

LAB 01

Domains, Subdomains & Data Assets

■ 45 min ■ ■ Beginner

■ Lab Objectives

- Create a new subdomain under the Revenue domain
- Register a Snowflake table as a Data Asset using Browse or manual entry
- Set ownership, description, and criticality
- Certify a dataset and understand the certification lifecycle
- Navigate the drill-down: Global → Domain → Subdomain → Table

---

Background

Everything in the platform is organized in a three-tier hierarchy: **Domain** → **Subdomain** → **Data Asset**. Quality scores roll up from table to subdomain to domain. Domain owners are restricted to their assigned domain — they cannot see data from other domains.

■ **Key Concept:** A Data Asset is a registered Snowflake table. The platform stores only the metadata (schema, table name, owner, description). It never copies your data.

Part 1: Create a Subdomain

Step 1: Navigate to Domain Management

- Go to: Administration → Domain Management
- You see 7 domain cards seeded at startup

Step 2: Add a subdomain to Revenue

- Click the '...' menu on the Revenue domain card → 'Manage Subdomains'
- Click '+ Add Subdomain'
- Subdomain Name: Invoicing
- Description: Invoice processing and accounts receivable tracking
- Owner Email: your-email@company.com
- Click Save
- ■ Expected: 'Invoicing' appears in Revenue's subdomain list

Part 2: Register a Snowflake Table

■ **Note:** Skip Step 3 if you don't have Snowflake. Use 'Enter Manually' in Step 4 with any schema/table name. The asset registers but rules will show 'error' when run.

Step 3: Configure a Snowflake Connection

- Go to: Administration → Settings → Snowflake → '+ Add Connection'
- Connection Name: Production DW
- Account: your-account (without .snowflakecomputing.com)

- User: your_service_user, Password: your_password
- Warehouse: DQ_EXECUTION_WH, Role: DQ_PLATFORM_ROLE
- Click 'Test Connection' (should show 'Connection successful'), then Save

Step 4: Register the table

- Go to: Data Quality → Data Assets → Register Table
- Browse tab: Connection → Database → Schema → pick your INVOICES table OR
- Manual tab: type sf_schema_name and sf_table_name directly
- Domain: Revenue, Subdomain: Invoicing
- Table Description: Stores all customer invoices including amounts, dates, and payment status
- Business Owner: Jane Smith / jane@company.com
- Criticality: Critical
- Click 'Register Table'
- ■ Expected: Asset appears with 'Uncertified' badge

Step 5: Certify the dataset

- Click Edit (pencil icon) on the new asset
- Certification Status: change to 'Warning'
- Click Save Changes
- ■ Expected: Yellow 'Warning' badge replaces 'Uncertified'

Part 3: Navigate the Hierarchy

Step 6: Drill down to the Table Dashboard

- Global Dashboard → click Revenue card → Domain Dashboard
- Click 'Invoicing' subdomain → Subdomain Dashboard
- Click your invoices table row → Table Dashboard
- Observe: breadcrumb shows Home → Revenue → Invoicing → invoices
- Notice: Quality Score ring shows 100% (no rules run yet)
- Notice: certification badge shows 'Warning' in yellow

■ **Lab 01 Complete!** Domain hierarchy and first table registered. Proceed to Lab 02.

LAB 02

Creating & Running Data Quality Rules

■ 60 min ■ ■ Beginner

■ Lab Objectives

- Create 5 rules across 5 different rule types
- Understand config for null_check, uniqueness, accepted_values, range_check, freshness_check
- Run rules manually and interpret passed/failed/error results
- Import a rule via JSON and go through the approval workflow
- Edit a rule and inspect version history — restore a prior version
- Use bulk operations to run all rules simultaneously

Background

Rules are the core engine. Each rule = one quality expectation on a column. On execution, the platform runs generated SQL against Snowflake and computes a quality score (0–100%). The platform supports 16 rule types — 12 standard SQL-based and 4 AI-powered semantic types.

Part 1: Create 5 Rules

Step 1: Create a null check rule

- Go to: Data Quality → Rules → '+ New Rule'
- Rule Name: invoice_id_not_null
- Description: Every invoice must have an ID — null IDs break downstream reporting
- Domain: Revenue, Subdomain: Invoicing, Table: invoices
- Rule Type: null_check
- Target Column: INVOICE_ID (or your PK column)
- Severity: Critical
- Leave Custom SQL blank (auto-generated)
- Click 'Create Rule'
- ■ Expected: Rule created with status 'Active'

Now create 4 more rules using this reference table:

Rule Name	Rule Type	Column	Config	Severity
invoice_id_unique	uniqueness_check	INVOICE_ID	None	Critical
valid_invoice_status	accepted_values_check	STATUS	accepted_values: ['PAID','PENDING','FAILED']	High
invoice_amount_positive	range_check	INVOICE_AMOUNT	min_value: 0	High
invoices_freshness	freshness_check	CREATED_AT	max_hours: 24	Medium

Part 2: Run Rules

Step 2: Run a single rule

- In the Rules list, find 'invoice_id_not_null', click ■ Run
- A spinner appears; wait 5–30 seconds (depends on table size)
- Result: green 'passed' + score, OR red 'failed' + score, OR orange 'error'
- Click 'view' next to the result to see run details
- In run detail: observe total_rows_scanned, failed_rows_count, executed_sql, quality_score

Step 3: Run all 5 rules at once

- In Rules list: check the checkbox on each rule row (or Select All at top)
- Bulk action toolbar appears at the bottom of the page
- Click 'Run All Selected'
- ■ Expected: All 5 rules appear in Execution Logs within 60 seconds

Step 4: Check the Table Dashboard

- Navigate to the Table Dashboard for your invoices table
- ■ Expected: Quality Score ring now shows a real score (not 100%)
- 30-day trend chart shows first data points
- Each rule row shows last run status

Part 3: Approval Workflow

Step 5: Import a rule via API

- Create import.json with this content:

```
{
  "domain": "Revenue",
  "subdomain": "Invoicing",
  "asset": {"sf_schema_name": "REVENUE_DW", "sf_table_name": "INVOICES"},
  "rules": [{
    "rule_name": "invoice_date_not_future",
    "rule_type": "business_rule_check",
    "severity": "medium",
    "config": {"condition": "INVOICE_DATE <= CURRENT_DATE()"},
    "rule_description": "Invoice dates must not be in the future"
  }]
}
```

```
curl -X POST http://localhost:8000/rules/import \
  -H "Content-Type: application/json" \
  -d @import.json
```

Step 6: Approve the imported rule

- Go to Rules → filter Status = 'Pending Review'
- Click 'Review' on the imported rule
- Yellow approval panel appears at top of Rule Detail page
- Click 'Approve'
- ■ Expected: Status changes to 'Active'

Part 4: Version History

Step 7: Edit and roll back a rule

- Click pencil icon on 'invoice_amount_positive'
- Change min_value from 0 to -1000 (to allow credit notes)
- Update Description: Allows negative amounts for credit/refund invoices
- Click 'Save Changes'
- Go to Rule Detail → click 'Version History' tab
- ■ Expected: v1 snapshot shown with original min_value: 0
- Click 'Restore' on v1 — rule returns to pending_review with original values

■ **Lab 02 Complete!** 5 rules created, run, approved, and versioned.

LAB 03

Scheduling & Automated Execution

■ 40 min ■ ■■
Intermediate

■ Lab Objectives

- Create a table-level daily schedule
- Create a rule-level hourly schedule override for a critical check
- Understand schedule priority: Rule > Table > Subdomain > Domain > Global
- Pause, resume, and trigger schedules manually
- Configure SLA thresholds and quality alerts

Background

Schedules run rules automatically. The most specific active schedule wins. A rule-level hourly schedule overrides a table-level daily schedule for that specific rule. You can have as many schedule levels as needed — the hierarchy always resolves to one winner per rule.

Part 1: Create a Daily Table Schedule

Step 1: Create the schedule

- Go to: Data Quality → Schedules → '+ New Schedule'
- Level: Table, Asset: select your invoices table
- Frequency: Daily, Run at: 06:00, Timezone: America/Los_Angeles
- Click Save
- ■ Expected: Schedule shows status 'Active' with next_run timestamp

Part 2: Add a Rule-Level Override

Step 2: Create an hourly override for the critical null check

- Click '+ New Schedule'
- Level: Rule, Rule: invoice_id_not_null
- Frequency: Hourly
- Click Save
- ■ Expected: invoice_id_not_null now runs every hour; other rules still run daily

■ **Key Concept:** Priority order: Rule > Table > Subdomain > Domain > Global. The most specific active schedule always wins. Pausing a table schedule does not affect rule-level schedules.

Part 3: Manual Operations

Step 3: Trigger the table schedule now

- In Schedules list, find your daily table schedule
- Click ■ Run Now
- Go to Execution Logs

- ■ Expected: All active rules for the table execute within 60 seconds

Step 4: Pause and resume

- Click the Pause button on the daily schedule
- ■ Expected: Status = 'Paused', next_run disappears
- Click Resume to re-activate

Part 4: SLA Thresholds

Step 5: Set a quality SLA for your table

- Go to: Administration → Settings → SLA & Quality
- Under 'Per-Entity SLA Configs' → '+ Add Config'
- Entity Type: Table, paste your asset_id (from URL when viewing Table Dashboard)
- Min Quality Score: 95, Max Failure %: 5
- Alert Email Recipients: your-email@company.com
- Click Create
- ■ Expected: Quality score below 95% now triggers an SLA breach event

■ **Lab 03 Complete!** Automated quality monitoring is now running.

LAB 04

AI Copilot — Rule Wizard & Natural Language Rules

■ 50 min ■ ■■
Intermediate

■ Lab Objectives

- Use the AI Copilot Rule Creation Wizard to create rules step-by-step
- Create a rule from a plain-English description using NL-to-rule
- Retrieve an AI failure explanation for a failing run
- Run Root Cause Analysis on a failed run
- Use the AI Assistant for platform-level questions
- Discover PII columns using the AI PII scanner

■ **Note:** This lab requires an LLM provider. Ollama (free, local) or OpenAI work well. Configure in Administration → Settings → LLM / AI and test before starting.

Part 1: Rule Creation Wizard

Step 1: Open AI Copilot

- Click the floating chat button (bottom-right corner of any page)
- Panel opens with two modes: Chat and Create Rule Wizard
- Click '+ Create Rule (Wizard)'

Step 2: Complete the 6-step wizard

- Step 1: Domain = Revenue → Subdomain = Invoicing
- Step 2: Table = select your invoices table
- Step 3: Columns = select CUSTOMER_EMAIL
- Step 4: Rule Type = regex_check
- Step 5: Click 'Generate SQL' — AI creates email validation regex
- Step 6: Severity = Medium → click 'Save Rule'
- ■ Expected: Rule created as 'pending_review' (AI rules always need approval)

Part 2: Natural Language Rule Creation

Step 3: Call the NL-to-Rule API

- Replace YOUR_ASSET_ID with your asset's ID (from URL when viewing Table Dashboard):

```
curl -X POST http://localhost:8000/ai/rules/from-natural-language \
  -H "Content-Type: application/json" \
  -d '{
    "description": "Invoice amounts must always be positive",
    "asset_id": "YOUR_ASSET_ID",
    "domain_context": "Revenue billing"
  }'
```

Step 4: Review the AI output

- ■ Expected: {"rule_type": "range_check", "target_column": "INVOICE_AMOUNT",

- "severity": "high", "rule_config": {"min_value": 0}, "suggested_sql": "..."}
- The AI chose the correct rule_type from your plain-English description
- You can paste this into the Create Rule form to save it

Part 3: AI Failure Explanation & RCA

Step 5: Get an AI explanation for a failure

- In Execution Logs, click any failed run to expand it
- Click '■ AI Explain' button
- ■ Expected: AI explains what failed, why it matters, root cause, and suggested fix

Step 6: Run Root Cause Analysis

- Find a failed run_id from Execution Logs
- curl -X POST http://localhost:8000/ai/rca/YOUR_RUN_ID
- ■ Expected: JSON with root_cause, explanation, confidence (0–1), recommended_action

Part 4: AI Assistant & PII Discovery

Step 7: Use the full-page AI Assistant

- Go to: AI Intelligence → AI Assistant
- Try these questions:
- "Which tables in Revenue have the most critical failures this week?"
- "Suggest 5 rules for an HR payroll table with columns: employee_id, salary, department, hire_date"
- "What is the current quality score for the Revenue domain?"

Step 8: Run PII Discovery scan

- curl -X POST http://localhost:8000/ai/discover-pii/YOUR_ASSET_ID
- ■ Expected: List of findings with column_name, pii_type, confidence, suggested_classification
- CUSTOMER_EMAIL should score ~0.95 as PII → email_address

■ **Lab 04 Complete!** AI features explored across rule creation, explanation, RCA, and PII discovery.

LAB 05

Data Catalog, Glossary & Column Profiling

■ 55 min ■ ■■
Intermediate

■ Lab Objectives

- Create business glossary terms and link them to tables and columns
- Apply sensitivity classifications (PII, SENSITIVE, CONFIDENTIAL)
- Trigger on-demand column profiling to capture statistics
- Create a Data Product bundling related tables
- Search the catalog and filter by domain, type, and classification

---

Part 1: Business Glossary

Step 1: Create glossary terms

- Go to: Catalog → Glossary → '+ New Term'
- Term Name: Annual Recurring Revenue (ARR)
- Definition: Value of recurring revenue normalized to one year. Calculated as MRR × 12.
- Examples: Customer pays $500/month → ARR = $6,000
- Synonyms: ARR, Annualized Revenue
- Domain: Revenue, Owner Email: finance@company.com
- Click Save. Create a second term: Invoice (with a business definition).

Step 2: Link the Invoice term to your table

- Click on the 'Invoice' term to open its detail page
- Click '+ Link to Asset', select your invoices table, leave Column Name blank
- Click Save
- ■ Expected: invoices table appears in the term's 'Linked Assets' section
- Navigate to Table Dashboard — 'Business Terms' section shows 'Invoice' linked

Part 2: Sensitivity Classifications

Step 3: Apply a PII classification via API

- Replace YOUR_ASSET_ID with your asset's ID:

```
curl -X POST "http://localhost:8000/assets/YOUR_ASSET_ID/classifications" \
  -H "Content-Type: application/json" \
  -d '{"column_name":"CUSTOMER_EMAIL","classification":"PII",
       "justification":"Customer email is PII under GDPR"}'
```

Step 4: Apply more classifications

- Apply SENSITIVE to INVOICE_AMOUNT (business-sensitive revenue data)
- View the summary: GET /classifications/summary
- ■ Expected: {"PII": 1, "SENSITIVE": 1}

- GET /classifications/pii-assets — lists all tables with PII labels

Part 3: Column Profiling

Step 5: Trigger profiling run

- curl -X POST http://localhost:8000/assets/YOUR_ASSET_ID/columns/profile
- ■ Expected: {"job_id": "...", "status": "queued"}
- Poll for completion (30–120 sec): GET /assets/YOUR_ASSET_ID/columns/profile/status

Step 6: Review profiling results

- GET /assets/YOUR_ASSET_ID/columns
- ■ Expected: Each column shows null_count, unique_count, min_value, max_value, cardinality_pct
- In Table Dashboard → Schema tab: column statistics now visible
- Columns with null_rate > 10% highlighted in orange — potential rule candidates

Part 4: Data Products

Step 7: Create a Revenue 360 Data Product

- Go to: Catalog → Data Products → '+ New Product'
- Product Name: Revenue 360, Domain: Revenue, Version: 1.0
- Description: Unified revenue product covering invoices, subscriptions, and billing
- Click Create, then '+ Add Table', select your invoices table, Role: primary
- ■ Expected: Product quality score aggregates across all linked tables

Part 5: Catalog Search

Step 8: Search across all entity types

- Go to: Catalog → Data Catalog
- Type 'invoice' in the search bar
- ■ Expected: Your table AND the 'Invoice' glossary term AND 'Revenue 360' product all appear
- Filter by Type: Assets only, then Domain: Revenue
- Sort by Quality Score — your table at top

■ **Lab 05 Complete!** Rich data catalog with terms, classifications, profiling, and data products built.

LAB 06

Governance Scorecards & Policy Engine

■ 60 min ■ ■■
Intermediate

■ Lab Objectives

- Interpret the 6-dimension governance scorecard
- Run the policy engine and review auto-generated violations
- Fix a violation and verify it resolves on next evaluation
- Set an announcement banner on a data asset
- Add collaboration comments (question and issue types)

Part 1: Governance Scorecard

Step 1: View the scorecard

- Go to: Governance → Governance Hub → Scorecards tab
- Find Revenue domain row
- Observe 6 dimensions: Quality (40%), Documentation (20%), Classification (15%), Ownership (10%), Certification (10%), SLA (5%)

Dimension	Weight	How to improve
Data Quality	40%	Run rules and fix failures — highest leverage dimension
Documentation	20%	Add table_description to all assets, column descriptions after profiling
Classification	15%	Apply PII/SENSITIVE labels — done in Lab 05
Ownership	10%	Set owner_email on every asset — done in Lab 01
Certification	10%	Move from 'Uncertified' to 'Certified' or 'Warning'
SLA Compliance	5%	Quality score above SLA threshold set in Lab 03

Step 2: Run policy evaluation

- In Scorecards tab → click '■ Run Evaluation'
- Wait 5–10 seconds
- ■ Expected: Violation count badge appears
- Switch to 'Violations' tab to see all open violations

Part 2: Fix a Violation

Step 3: Understand the 4 built-in policies

- owner_required: asset has no owner_email → Medium severity
- certification_required: asset still Uncertified → Low severity
- no_rules_defined: asset has zero active rules → High severity
- stale_description: asset has no table_description → Low severity

Step 4: Fix an owner_required violation

- In Violations tab, find an owner_required violation for any asset
- Click the asset name to navigate to Data Assets
- Click Edit — add an owner_email value — Save Changes
- Return to Governance Hub → click 'Run Evaluation' again
- ■ Expected: The owner_required violation for that asset is now 'resolved'

Part 3: Announcements & Collaboration

Step 5: Create a warning announcement on your table

```
curl -X POST "http://localhost:8000/announcements" \
  -H "Content-Type: application/json" \
  -d '{
    "entity_type": "asset",
    "entity_id": "YOUR_ASSET_ID",
    "title": "Schema change planned for 2026-06-01",
    "body": "INVOICE_STATUS will be renamed to STATUS. Update your rules.",
    "announcement_type": "warning",
    "expires_at": "2026-06-15T00:00:00"
  }'
```

Step 6: Add a collaboration comment

```
curl -X POST "http://localhost:8000/comments" \
  -H "Content-Type: application/json" \
  -d '{"entity_type":"asset","entity_id":"YOUR_ASSET_ID",
       "body":"Why no volume_check on this table? Critical for billing.",
       "comment_type":"question"}'
```

■ Expected: Yellow banner appears on Table Dashboard. Question comment is visible on Discussion tab.

■ **Lab 06 Complete!** Governance scorecards, policy violations, and collaboration features explored.

LAB 07

Compliance Frameworks & Data Contracts

■ 60 min ■ ■■■
Advanced

■ Lab Objectives

- Map existing DQ rules to GDPR compliance requirements
- Run a compliance assessment and identify gaps
- Retrieve a compliance evidence package for an audit
- Create a data contract with quality and schema SLAs
- Validate the contract against current quality scores

Part 1: Explore Compliance Frameworks

Step 1: List available frameworks

- GET http://localhost:8000/compliance/frameworks
- ■ Expected: 6 frameworks — GDPR, CCPA, HIPAA, SOX, BCBS 239, ISO 27001
- Note the GDPR framework_id from the response (a UUID)

Step 2: List GDPR requirements

- GET http://localhost:8000/compliance/frameworks/GDPR_ID/requirements
- Each requirement maps to one or more DQ rule types
- Example: 'Data Accuracy' maps to null_check, range_check, regex_check

Part 2: Run Assessment & Find Gaps

Step 3: Assess invoices table against GDPR

- POST http://localhost:8000/compliance/frameworks/GDPR_ID/assess/YOUR_ASSET_ID

```
curl -X POST "http://localhost:8000/compliance/frameworks/GDPR_ID/assess/ASSET_ID"
```

Step 4: Review the response

- ■ Expected: {"total": 8, "gaps": 3, "requirements": [
- {"req_code":"GDPR_5_1_d","req_name":"Data Accuracy","status":"compliant"},
- {"req_code":"GDPR_5_1_e","req_name":"Storage Limitation","status":"gap"}, ...]}
- Requirements with status='compliant' have matching rules with recent passing runs
- Requirements with status='gap' are your compliance action items

Step 5: Export evidence for a compliant requirement

- From the assessment response, note a mapping_id for a 'compliant' requirement
- GET http://localhost:8000/compliance/evidence/MAPPING_ID
- ■ Expected: Evidence package with rule definition + last 5 run results
- This package can be used as audit evidence for GDPR/SOX reviews

Part 3: Create a Data Contract

Step 6: Create the contract

```
curl -X POST "http://localhost:8000/contracts" \
  -H "Content-Type: application/json" \
  -d '{
    "asset_id": "YOUR_ASSET_ID",
    "contract_name": "Revenue Invoices SLA v1.0",
    "producer_team": "Data Engineering",
    "consumer_team": "Finance Analytics",
    "min_quality_score": 95.0,
    "max_staleness_hours": 24,
    "sla_description": "Invoice table: 95%+ quality, updated daily by 6 AM PST"
  }'
```

Step 7: Validate the contract

- POST http://localhost:8000/contracts/CONTRACT_ID/validate
- ■ Expected: {"compliant": true/false, "issues": ["list of problems if any"]}
- If quality score < 95: contract is non-compliant
- In UI: Governance → Data Contracts — your contract now appears

■ **Lab 07 Complete!** Compliance assessment run, evidence exported, data contract created.

LAB 08

Privacy Engineering & Masking Policies

■ 45 min ■ ■■■
Advanced

■ Lab Objectives

- Create column-level masking policies for PII data
- Apply all 5 masking types and understand when to use each
- Run the PII exposure report to find unprotected sensitive data
- Fix exposure by adding masking policies and re-running the report

---

■■ **Warning:** Masking policies control what the DQ platform shows in sample records. They do NOT affect your actual Snowflake data. For database-level masking, use Snowflake Dynamic Data Masking separately.

Part 1: Create Masking Policies

Step 1: Create a full_mask for email

```
curl -X POST "http://localhost:8000/privacy/masking-policies" \
  -H "Content-Type: application/json" \
  -d '{
    "asset_id": "YOUR_ASSET_ID",
    "column_name": "CUSTOMER_EMAIL",
    "masking_type": "full_mask",
    "applies_to_roles": "viewer,auditor",
    "unmasked_roles": "admin,domain_owner,data_owner"
  }'
```

Masking type reference:

Type	Input	Output	Best For
full_mask	jane@co.com	****@*****.***	Emails, usernames
partial_mask	4111-1111-1111-1234	****-****-****-1234	Credit cards
hash	john@co.com	a3f8b2c1... (SHA-256)	Join keys (consistent, irreversible)
tokenize	John Doe	Xkp7 Qmt3	Names (length-preserving, reversible for privileged)
nullify	192.168.1.100	NULL	IP addresses, maximum privacy

Part 2: PII Exposure Report

Step 2: Run the exposure report

- GET http://localhost:8000/privacy/pii-exposure-report
- Shows tables with PII/SENSITIVE classifications but NO masking policy
- ■ Expected: If you added classifications in Lab 05 without masking, those tables appear

Step 3: Fix the exposure

- For each table in the report, create masking policies for all PII columns

- Re-run the exposure report
- ■ Expected: 0 unprotected tables once all PII columns have masking policies

Step 4: View masking summary

- GET http://localhost:8000/privacy/assets/YOUR_ASSET_ID/masking-summary
- ■ Expected: {"masked_column_count": 1, "policies": [...]}
- In Table Dashboard → Schema tab: ■ icon shows next to masked columns

■ **Lab 08 Complete!** Masking policies created and PII exposure eliminated.

LAB 09

Incident Management & Root Cause Analysis

■ 55 min ■ ■■■
Advanced

■ Lab Objectives

- Create a data quality incident and move through the lifecycle
- Configure an on-call schedule for domain-based routing
- Attach a runbook to a critical rule
- Generate an AI post-mortem for a resolved incident
- Create an anomaly detector and run a Z-score detection

Part 1: Incident Lifecycle

Step 1: Create a manual incident

- Go to: Governance → Incidents → '+ New Incident'
- Asset: invoices table, Title: Invoice batch validation failures 2026-05-13
- Severity: High, Click Create
- ■ Expected: Incident status = 'Open', MTTD timer starts

Step 2: Progress through lifecycle

- Click the incident → Click 'Start Investigation' → status = investigating
- After reviewing, click 'Resolve'
- ■ Expected: status = resolved, ttr_minutes auto-calculated

Part 2: On-Call Routing

Step 3: Create an on-call schedule for Revenue

```
curl -X POST "http://localhost:8000/oncall" \
  -H "Content-Type: application/json" \
  -d '{"domain_id":"REVENUE_DOMAIN_ID","oncall_email":"your@email.com",
       "oncall_slack":"#revenue-data-quality",
       "effective_from":"2026-01-01T00:00:00",
       "effective_until":"2026-12-31T23:59:59","timezone":"UTC"}'
```

Part 3: Runbook

Step 4: Attach a runbook to the null check rule

- Get your rule_id: GET /rules?rule_name=invoice_id_not_null

```
curl -X POST "http://localhost:8000/runbooks" \
  -H "Content-Type: application/json" \
  -d '{"rule_id":"YOUR_RULE_ID",
       "title":"Invoice ID Null Failure Runbook",
       "steps":"1. Check ETL pipeline in Airflow\n2. Look for failed invoice_loader DAG\
n3. Check source system for records without ID\n4. If ETL failed: rerun affected
partition\n5. If source issue: escalate to Finance Ops",
       "escalation_path":"L1: Data Eng on-call → L2: Lead → L3: VP Eng"}'
```

Part 4: AI Post-Mortem

Step 5: Generate post-mortem for the resolved incident

- POST http://localhost:8000/ai/incidents/YOUR_INCIDENT_ID/generate-postmortem
- ■ Expected: AI generates Markdown post-mortem with: executive summary, timeline,
- root cause, contributing factors, and action items

Part 5: Anomaly Detection

Step 6: Create and run a Z-score anomaly detector

```
# Create detector
curl -X POST "http://localhost:8000/anomaly/detectors" \
  -H "Content-Type: application/json" \
  -d '{"asset_id":"YOUR_ASSET_ID","detector_type":"zscore","config":{"z_threshold":2.5}}'

# Run it (DETECTOR_ID from above response)
curl -X POST "http://localhost:8000/anomaly/detectors/DETECTOR_ID/run"
```

■ Expected: If quality scores have varied significantly, an anomaly detection record is created. Response shows: {"anomaly_found": true/false, "z_score": 2.8, "observed": 87.3, "mean": 96.1}

■ **Lab 09 Complete!** Incident lifecycle, on-call routing, runbooks, post-mortem, and anomaly detection explored.

LAB 10

Data Lineage & Impact Analysis

■ 50 min ■ ■■■
Advanced

■ Lab Objectives

- Register upstream and downstream lineage for a table
- Mark a downstream as critical to trigger HIGH blast radius
- View the impact analysis and understand blast radius scoring
- Upload a dbt manifest to auto-populate lineage (if using dbt)
- Create a cross-table semantic consistency rule

Part 1: Register Lineage

Step 1: Register a non-critical downstream

```
curl -X POST "http://localhost:8000/assets/YOUR_ASSET_ID/lineage" \
  -H "Content-Type: application/json" \
  -d '{"upstream_asset_id":"YOUR_ASSET_ID","downstream_name":"finance_reporting.
monthly_revenue",
       "downstream_type":"snowflake_table","lineage_type":"table_to_table","is_critical":
false}'
```

Step 2: Register a critical downstream (BI dashboard)

```
curl -X POST "http://localhost:8000/assets/YOUR_ASSET_ID/lineage" \
  -H "Content-Type: application/json" \
  -d '{"upstream_asset_id":"YOUR_ASSET_ID","downstream_name":"CFO Revenue Dashboard -
Looker",
       "downstream_type":"looker_dashboard","is_critical":true}'
```

Part 2: Impact Analysis

Step 3: Get blast radius

- GET http://localhost:8000/assets/YOUR_ASSET_ID/lineage/impact
- ■ Expected: blast_radius_score = "HIGH" because is_critical=true on a downstream
- downstream_count = 2 (both your registrations)

Blast Radius	Trigger	Meaning
HIGH	3+ downstream OR any single is_critical=true	CFO/critical reports at risk — escalate immediately
MEDIUM	Downstream consumers exist, none critical	Impact exists but not executive-level — investigate
LOW	No registered downstream dependencies	Isolated table — lower priority to fix

Part 3: dbt Integration (optional)

Step 4: Upload dbt manifest

- In your dbt project: dbt compile (generates target/manifest.json and target/catalog.json)

```
curl -X POST "http://localhost:8000/integrations/dbt/upload" \
  -F "manifest=@target/manifest.json" \
  -F "catalog=@target/catalog.json"
```

■ Expected: dbt ref() relationships populate data_lineage. Model descriptions populate table_description. Column docs populate column_metadata.description.

Part 4: Semantic Cross-Table Rule

Step 5: Create a referential sanity check

- Rules → New Rule → Rule Type: referential_sanity_check
- Condition: PAYMENT_DATE IS NULL OR PAYMENT_DATE >= INVOICE_DATE
- Description: Payment cannot occur before the invoice was issued
- Severity: High
- Click Create and run it
- ■ Expected: SQL checks the business logic across columns in the same table

■ **Lab 10 Complete!** Lineage registered, blast radius = HIGH, dbt integration explored.

LAB 11

Rule Marketplace & Template Publishing

■ 40 min ■ ■■
Intermediate

■ Lab Objectives

- Seed the marketplace with example templates
- Get AI-recommended templates matched to your table's columns
- Import a template as a draft rule with one API call
- Create and publish your own rule template
- Rate a community template

■ **Note:** The marketplace starts empty. This lab seeds it with example templates. In production, templates accumulate as users publish them.

Step 1: Seed an example template

```
curl -X POST "http://localhost:8000/marketplace/templates" \
  -H "Content-Type: application/json" \
  -d '{"template_name":"Invoice ID Not Null (Finance Standard)",
       "description":"Every invoice must have a non-null invoice ID. Required for GL
reconciliation.",
       "rule_type":"null_check","target_domains":"Revenue,Finance",
       "target_industries":"Finance,E-commerce","is_public":true}'
```

Repeat with 2–3 more templates (range_check for amounts, freshness_check for ETL SLA, regex_check for email).

Step 2: Get AI-recommended templates

- GET http://localhost:8000/marketplace/templates/recommended?asset_id=YOUR_ASSET_ID
- ■ Expected: AI scores each template 0–1 based on your column context
- Template named 'Invoice ID Not Null' should score ~0.95 if your column names match

Step 3: Import a template as a draft rule

- Note the template_id from: GET /marketplace/templates

```
curl -X POST "http://localhost:8000/marketplace/templates/TEMPLATE_ID/import" \
  -H "Content-Type: application/json" \
  -d '{"asset_id":"YOUR_ASSET_ID","severity":"critical"}'
```

Step 4: Approve the imported rule

- Go to Rules → filter Status = Pending Review
- Click Review → Approve
- ■ Expected: Rule is now Active — ready to run

Step 5: Publish your own template

- Create a POST /marketplace/templates with a rule you created earlier
- Set is_public=true to share it
- Rate another template: POST /marketplace/templates/TEMPLATE_ID/rate with {"rating": 5}

■ **Lab 11 Complete!** Marketplace seeded, templates recommended, imported, and published.

LAB 12

API Automation & CI/CD Integration

■ 60 min ■ ■■■
Advanced

■ Lab Objectives

- Create a service account (machine-to-machine API key authentication)
- Authenticate all requests with X-API-Key header
- Build a quality gate shell script that blocks bad pipeline deployments
- Use bulk execute and poll for async job completion
- Set up a GitHub Actions workflow template

Part 1: Service Account

Step 1: Create a CI/CD service account

```
# Get admin JWT first
TOKEN=$(curl -s -X POST http://localhost:8000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@example.com","password":"admin123"}' \
  | python3 -c "import sys,json; print(json.load(sys.stdin)['access_token'])")

# Create service account
curl -X POST http://localhost:8000/service-accounts \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name":"github-actions-pipeline","description":"Used by CI/CD","role":
"data_owner"}'
```

■ **Critical:** The api_key in the response is shown ONCE. Copy it to a secure password manager or GitHub Secret immediately. It cannot be retrieved again — only rotated.

Step 2: Verify API key authentication

- export DQ_API_KEY='sa_YOURPREFIX_YOURSECRET'
- curl http://localhost:8000/rules -H 'X-API-Key: '$DQ_API_KEY
- ■ Expected: JSON array of rules — authenticated as the service account

Part 2: Quality Gate Script

Step 3: Create quality_gate.sh

```
#!/bin/bash
# quality_gate.sh — Run before deploying any pipeline touching this table
set -e
DQ_URL="${DQ_PLATFORM_URL:-http://localhost:8000}"
ASSET_ID="${1}"
MIN_SCORE="${2:-95}"

echo "Running quality gate for asset $ASSET_ID (min score: $MIN_SCORE%)"

# Execute all active rules for the table
curl -s -X POST "$DQ_URL/execute/table/$ASSET_ID/sync" \
  -H "X-API-Key: $DQ_API_KEY" > /tmp/exec.json
```

```
# Evaluate the quality gate
RESULT=$(curl -s -X POST "$DQ_URL/cicd/gate/evaluate" \
  -H "X-API-Key: $DQ_API_KEY" \
  -H "Content-Type: application/json" \
  -d "{"asset_id":"$ASSET_ID","min_quality_score":$MIN_SCORE,"fail_on_critical":true}")

PASSED=$(echo $RESULT | python3 -c "import sys,json; print(json.load(sys.stdin)[
'gate_passed'])")
SCORE=$(echo  $RESULT | python3 -c "import sys,json; print(json.load(sys.stdin).get(
'quality_score',0))")

echo "Quality score: $SCORE%"
if [ "$PASSED" = "True" ]; then echo "■ PASSED"; exit 0
else echo "■ FAILED — blocking deployment"; exit 1; fi
```

Step 4: Run the quality gate

- chmod +x quality_gate.sh
- export DQ_API_KEY='sa_...' (your service account key)
- ./quality_gate.sh YOUR_ASSET_ID 95
- ■ Expected: '■ PASSED — safe to deploy' OR '■ FAILED' with blocking failures listed

Part 3: Async Bulk Execute with Job Polling

Step 5: Execute multiple rules asynchronously

```
# Get rule IDs for your 5 rules (note them from GET /rules)
curl -X POST "http://localhost:8000/rules/bulk/execute" \
  -H "X-API-Key: $DQ_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"rule_ids":["RULE_ID_1","RULE_ID_2","RULE_ID_3","RULE_ID_4","RULE_ID_5"]}'
# Returns: {"job_id":"...","status":"queued","total":5}
```

Step 6: Poll for completion

- GET http://localhost:8000/rules/bulk/jobs/JOB_ID every 5 seconds
- ■ Expected: queued → running → {"status":"completed","completed":5,"failed":0}

Part 4: GitHub Actions Template

```
# .github/workflows/data-quality.yml
name: Data Quality Gate
on:
  push:
    branches: [main]
    paths: ['pipelines/revenue/**']
jobs:
  quality-gate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run DQ Gate
        env:
          DQ_PLATFORM_URL: ${{ secrets.DQ_PLATFORM_URL }}
          DQ_API_KEY: ${{ secrets.DQ_API_KEY }}
        run: |
          chmod +x quality_gate.sh
          ./quality_gate.sh ${{ secrets.INVOICES_ASSET_ID }} 95
```

■ **Lab 12 Complete!** Service account created, quality gate built, CI/CD workflow template ready.

Appendix A: Quick API Reference

Most commonly used endpoints across all 13 labs. Full reference at http://localhost:8000/docs

Category	Method	Endpoint	Description
Auth	POST	/auth/login	Login → returns JWT access_token
Domains	GET	/domains	List all active domains
Assets	GET	/assets/enriched	Assets with domain/subdomain metadata
Assets	POST	/assets	Register a new table
Rules	GET	/rules/enriched	Rules with full metadata context
Rules	POST	/rules	Create a new rule
Rules	POST	/rules/{id}/approve	Approve a pending_review rule
Execute	POST	/execute/rule/{id}/sync	Run rule, wait for result
Execute	POST	/rules/bulk/execute	Run many rules → returns job_id
Execute	GET	/rules/bulk/jobs/{id}	Poll bulk job status
Runs	GET	/runs/enriched	Execution history with full context
AI	POST	/ai/rules/from-natural-language	Convert English → rule definition
AI	POST	/ai/rca/{run_id}	Root cause analysis for a failed run
AI	POST	/ai/discover-pii/{asset_id}	Scan columns for PII patterns
Catalog	GET	/catalog/search?q=invoice	Search assets, terms, products
Compliance	POST	/compliance/frameworks/{f}/assess/{a}	Run compliance assessment
Compliance	GET	/compliance/evidence/{mapping_id}	Export audit evidence package
Gov	GET	/governance/scorecards	Per-domain governance score
Gov	POST	/governance/policies/evaluate	Trigger policy evaluation
Contracts	POST	/contracts/{id}/validate	Check contract compliance now
Privacy	POST	/privacy/masking-policies	Create a masking policy
Privacy	GET	/privacy/pii-exposure-report	Find unprotected PII tables
Incidents	POST	/incidents	Create an incident
Anomaly	POST	/anomaly/detectors/{id}/run	Run Z-score detector
Lineage	GET	/assets/{id}/lineage/impact	Blast radius analysis
Marketplace	GET	/marketplace/templates/recommended	AI-matched templates
Mesh	GET	/mesh/topology	Cross-domain dependency graph
CICD	POST	/cicd/gate/evaluate	Quality gate check for CI/CD

Observ.	GET	/observability/freshness-board	SLA freshness for all tables
Observ.	GET	/observability/events/stream	SSE real-time event stream

Appendix B: Rule Type Configuration Reference

Rule Type	Required Config	Optional Config	Pattern
null_check	target_column	columns: [list]	WHERE col IS NULL
uniqueness_check	target_column	columns: [list]	GROUP BY HAVING COUNT>1
accepted_values_check	target_column, accepted_values:[...]		WHERE col NOT IN(...)
range_check	target_column	min_value, max_value	WHERE col<min OR col>max
freshness_check	target_column (date col)	max_hours (default 24)	DATEDIFF(h,MAX(col),NOW())>N
volume_check	-	min_rows, max_rows, date_column	COUNT(*) vs [min,max]
schema_drift_check	expected_columns:[...]	-	INFORMATION_SCHEMA check
referential_integrity_check	target_column, reference_table	reference_column	LEFT JOIN WHERE parent IS NULL
regex_check	target_column, pattern	-	NOT REGEXP_LIKE(col,pattern)
business_rule_check	condition (SQL WHERE)	-	WHERE NOT (condition)
custom_sql_check	sql (returns failed_count)	-	Custom SQL
semantic_consistency_check	condition (NL description)	columns:[list]	WHERE NOT (NL→SQL)
business_metric_check	metric_sql, min_value or max_value		CASE WHEN metric out of range
distribution_consistency_check	baseline_mean	tolerance_pct (default 20)	ABS(AVG(col)-baseline)>tol
llm_semantic_check	validation_prompt	sample_size (default 100)	SELECT * TABLESAMPLE N

Appendix C: Troubleshooting

Problem: Rule shows 'error' status

1. Settings → Snowflake → Test Connection
2. RESUME warehouse: ALTER WAREHOUSE DQ_EXECUTION_WH RESUME
3. Check logs: docker compose logs api | grep ERROR

Problem: AI returns 'LLM unavailable'

1. curl http://localhost:11434/api/tags (Ollama running?)
2. ollama pull qwen2.5:7b-instruct
3. In Docker: use http://host.docker.internal:11434 (not localhost)
4. Check OLLAMA_BASE_URL in .env matches your server

Problem: Frontend shows blank page

1. docker compose logs frontend | head -50
2. curl http://localhost:3000 — should return HTML
3. Check NEXT_PUBLIC_API_URL=http://localhost:8000 in frontend/.env.local

Problem: Quality score stays at 100% after running rules

1. Check Execution Logs for the run
2. status=error: Snowflake connection issue
3. status=passed with 0 failed rows: rule may be too lenient
4. Inspect executed_sql in run detail — is the SQL correct?

Problem: Service account key rejected

1. Key format must be: sa_XXXXXXXX_YYYYYY (8-char prefix + 32-char secret)
2. Header must be: -H 'X-API-Key: sa_...'
3. Check is_active: GET /service-accounts with admin JWT
4. Key is case-sensitive — check copy/paste

Problem: Governance scorecard shows 0 everywhere

1. Expected on first run with no data
2. Run rules → populates quality dimension
3. Add owner_email to assets → ownership dimension improves
4. Add table_description → documentation dimension improves

Appendix D: Lab Completion Checklist

Track your progress through all 13 labs:

- ■ Lab 00: Environment running, health=OK, LLM provider tested and working
- ■ Lab 01: Domain hierarchy created, table registered, certification status set
- ■ Lab 02: 5 rules across 4+ rule types, at least 1 run executed, approval workflow done
- ■ Lab 03: Daily schedule + hourly override created, manual trigger tested, SLA set
- ■ Lab 04: AI Copilot wizard used, NL-to-rule tested, AI explanation retrieved
- ■ Lab 05: Glossary term linked to table, PII classification applied, profiling run done
- ■ Lab 06: Scorecard viewed, policy violations resolved, announcement banner visible
- ■ Lab 07: Compliance assessment run, evidence exported, data contract created
- ■ Lab 08: Masking policies for PII columns, PII exposure report shows 0 gaps
- ■ Lab 09: Incident created and resolved, runbook attached, AI post-mortem generated
- ■ Lab 10: Upstream and downstream lineage registered, blast_radius=HIGH confirmed
- ■ Lab 11: Template imported, own template published with is_public=true
- ■ Lab 12: Service account created, quality gate script runs and blocks bad deploys

---

Data Quality & Governance Platform — Hands-On Lab Guide · v3.0 · May 2026
Total estimated time: 10.5 hours · Self-paced · All labs self-contained